Lab Sheet 3: Data Hazards on the Pipelined RV32I Simulator

1. Objectives

By the end of this lab your pipeline will compute the same results as the single-cycle engine on every test program, and will do it in far fewer cycles than your Lab 2 pipeline. You will:

1. Classify a read-after-write (RAW) dependence by its **distance** and predict how many cycles it costs.
2. Build a **hazard detection unit** in ID that stalls the pipeline, and understand the three things a stall must do.
3. Build a **forwarding (bypass) unit** that feeds EX from the EX/MEM and MEM/WB latches, removing almost every stall.
4. Build **load-use detection**, the one data hazard forwarding cannot remove, and explain why one stall is unavoidable.
5. Instrument the pipeline and explain the measured stall counts and CPI of both policies.

2. What You Start From

You need your working Lab 2 pipeline: `pipeline/control.{h,c}`, `pipeline/pipeline_state.h`, `pipeline/pipeline.c` with the snapshot/commit shell, and the `--pipeline` option in `main.c`. Check it before writing any new code:

```
cd riscv_sim_base
make
./rvsim tests/sample1_sum_loop.s --batch      # oracle: prints 55, cycles = 53
./rvsim tests/sample1_sum_loop.s --pipeline   # cycles = 108, retired = 53, stalls =
40, flushes = 11
```

If the pipeline figures do not match, fix Lab 2 first. Everything below is measured against them.

Rules of engagement

- `core/`, `isa/`, `assembler/` and `ui/` stay untouched. Read them freely.
- All new code goes in `pipeline/`. The single-cycle engine remains your **reference oracle**: your pipeline must leave the register file in exactly the same final state, in every mode.
- **Keep the Lab 2 stalling policy selectable.** `--pipeline` keeps stalling only; the forwarding pipeline gets a new option `--pipeline-forward`. The whole point of the lab is the comparison between the two, so do not delete the slow one.
- You may add one `int forwarding;` field to `Pipeline` and one flag in `main.c`. Nothing else changes outside `pipeline/`.

Files for this lab

```
riscv_sim_base/
+-- pipeline/
|   +-- hazard.h              Task 1  HazardVerdict, Operands, prototypes
|   +-- hazard.c              Tasks 1-3  hazard_check(), forward_operands()
|   +-- pipeline.c            edited: ID calls hazard_check(), EX calls
```

```
forward_operands()
|   +-- pipeline_state.h      edited: new counters + the forwarding flag
+-- tests/sample4_data_hazards.s   (Appendix A - copy it into tests/)
+-- tests/micro.s                  (Section 4 - five lines, for hand tracing)
```

3. Background: Where the Value Is Not Yet

A data hazard is a timing problem, not a logic problem. Consider two instructions:

```
addi t0, zero, 5     # producer: writes t0
addi t1, t0, 10      # consumer: reads t0
```

The producer writes the register file in **WB**. The consumer reads the register file in **ID**. In a five-stage pipeline those two events are three cycles apart in the wrong direction:

```
cycle ->         1     2     3     4     5     6
addi t0          IF    ID    EX    MEM   WB          <- t0 written here
addi t1, t0            IF    ID    EX    MEM   WB
                             ^
                             t0 read here: three cycles too early
```

The consumer reads a stale `t0` and the program silently computes the wrong answer. This is a **RAW (read-after-write)** hazard, and it is the only kind of data hazard this pipeline can suffer.

The hazard window is exactly three latches

When an instruction is in ID, the three instructions ahead of it sit in ID/EX, EX/MEM and MEM/WB. Those are precisely the instructions that may owe it a value. Anything older has already written back; anything younger cannot be its producer. So a hazard detector never looks further than three latches, and the cost of a dependence depends only on how far apart the two instructions are:

Distance (consumer - producer)	Producer is in	Stalls needed without forwarding
1 (back to back)	EX	3
2	MEM	2
3	WB	1
4 or more	retired	0

Distance 3 costs only one stall because WB writes the register file in the same cycle ID reads it. In your `pipeline_cycle()` the WB stage runs before ID calls `cpu_read_reg()`, which models a register file written in the first half of the cycle and read in the second half.

Three things that are never hazards

- A producer whose `rd` is `x0`. The write is discarded, so there is nothing to wait for.
- A producer whose `ctrl.reg_write` is 0: stores, branches and `ecall` write no register.
- An invalid latch. A bubble left by a stall or a flush owes nobody anything.

Missing any of these three turns a working pipeline into one that stalls forever.

Why WAR and WAW cannot happen here

Every instruction reads its registers in ID and writes its register in WB, and instructions pass through both stages strictly in program order. A younger instruction therefore can never write a register before an older one reads it (no WAR) or write the same register before an older one writes it (no WAW). Out-of-order issue, multiple write ports or variable-latency functional units would break that guarantee - which is why those machines need register renaming and this one does not.

4. Worked Example: Five Instructions, Two Policies

Put this in `tests/micro.s`. It is small enough to trace by hand and contains two distance-1 dependences.

```
.text
.globl main
main:
    addi t0, zero, 5        # I1  @0x00
    addi t1, t0, 10         # I2  @0x04   RAW on t0, distance 1
    add  t2, t1, t0         # I3  @0x08   RAW on t1 (d=1) and t0 (d=2)
    addi a7, zero, 10       # I4  @0x0c
    ecall                   # I5  @0x10
```

With stalling only (what you have now): 15 cycles, 6 stalls, CPI 3.00

```
cycle:    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
IF       I1   I2   I2*  I2*  I2*  I3   I3*  I3*  I3*  I4   I5    -    -    -    -
ID        -   I1   I2   I2   I2   I2   I3   I3   I3   I3   I4   I5    -    -    -
EX        -    -   I1    -    -    -   I2    -    -    -   I3   I4   I5    -    -
MEM       -    -    -   I1    -    -    -   I2    -    -    -   I3   I4   I5    -
WB        -    -    -    -   I1    -    -    -   I2    -    -    -   I3   I4   I5
                   <-- stall -->        <-- stall -->
```

Read it this way: in cycle 3 the consumer I2 is in ID and its producer I1 is in EX, so I2 is held in ID and a bubble is sent into EX. `I2*` marks a frozen fetch - the same PC is fetched again, because IF must not run ahead while ID is held. I2 only moves on in cycle 6, once I1 has left all three latches. Each distance-1 dependence costs 3 cycles, and there are two of them: 6 stalls.

With forwarding: 9 cycles, 0 stalls, CPI 1.80 - the ideal pipeline

```
cycle:    1    2    3    4    5    6    7    8    9
IF       I1   I2   I3   I4   I5    -    -    -    -
ID        -   I1   I2   I3   I4   I5    -    -    -
EX        -    -   I1   I2   I3   I4   I5    -    -
MEM       -    -    -   I1   I2   I3   I4   I5    -
WB        -    -    -    -   I1   I2   I3   I4   I5

cycle 4:  I2 in EX needs t0. I1 is in MEM, so EX/MEM.alu_result -> EX.
cycle 5:  I3 in EX needs t1 (I2 is in MEM:  EX/MEM -> EX)
                     and  t0 (I1 is in WB:  MEM/WB -> EX).
```

Nine cycles is `5 instructions + 4 fill cycles`, exactly the ideal figure from the Lab 2 appendix. The dependences did not disappear; the values were simply read from the latch that already holds them instead of from a register file that has not been written yet.

5. Task 1 - The Hazard Detection Unit

Files: `pipeline/hazard.h`, `pipeline/hazard.c`

Move the hazard logic out of `pipeline.c` into a unit of its own, the way the hardware has one. In `hazard.h` declare:

```
#include "pipeline_state.h"

typedef struct {
    int stall;      /* hold ID, bubble EX, freeze IF                */
    int load_use;   /* the producer is a load still in EX (Task 3)   */
} HazardVerdict;

HazardVerdict hazard_check(const Pipeline *p, const DecodedInstr *d,
                           const ControlSignals *c);
```

In `hazard.c`, one helper expresses the whole "is this latch my producer?" question. Write it once and use it for all three latches:

```
/* valid latch + really writes a register + not x0 + same register number */
static int produces(int valid, const ControlSignals *lc, uint32_t rd, uint32_t rs) {
    return valid && lc->reg_write && rd != 0 && rd == rs;
}
```

The stalling policy is then four calls per source register:

```
HazardVerdict hazard_check(const Pipeline *p, const DecodedInstr *d,
                           const ControlSignals *c) {
    HazardVerdict v = {0, 0};

    if (!p->forwarding) {                    /* Task 1: stall on any of 3 latches */
        uint32_t s1 = c->uses_rs1 ? d->rs1 : 0;
        uint32_t s2 = c->uses_rs2 ? d->rs2 : 0;
        if (produces(p->id_ex.valid,  &p->id_ex.ctrl,  p->id_ex.instr.rd,  s1) ||
            produces(p->id_ex.valid,  &p->id_ex.ctrl,  p->id_ex.instr.rd,  s2) ||
            produces(p->ex_mem.valid, &p->ex_mem.ctrl, p->ex_mem.instr.rd, s1) ||
            produces(p->ex_mem.valid, &p->ex_mem.ctrl, p->ex_mem.instr.rd, s2) ||
            produces(p->mem_wb.valid, &p->mem_wb.ctrl, p->mem_wb.instr.rd, s1) ||
            produces(p->mem_wb.valid, &p->mem_wb.ctrl, p->mem_wb.instr.rd, s2))
            v.stall = 1;
        return v;
    }

    /* Task 3 goes here. */
    return v;
}
```

Note the `c->uses_rs1 ? d->rs1 : 0` trick: an unused source register becomes `x0`, which `produces()` already rejects. Without it, `lui a0, 0x10` appears to read whatever junk sits in the `rs1` field of a U-type encoding and stalls for no reason.

A stall is three actions, not one

In the ID stage of `pipeline_cycle()`, a stall verdict must do all three of these. Dropping any one of them is the single most common bug in this lab:

```
HazardVerdict hz = hazard_check(p, &d, &c);
if (hz.stall) {
    nx.id_ex.valid = 0;      /* 1. a bubble enters EX this cycle          */
    nx.if_id       = p->if_id; /* 2. IF/ID is frozen: the consumer is      */
                            /*    re-decoded next cycle                  */
    stall = 1;              /* 3. local flag: IF must not fetch, and     */
    nx.stalls++;            /*    cpu->pc must not advance               */
}
```

Action 3 is enforced by the IF stage you already wrote: wrap the whole fetch in `if (!stall) { ... }`. If IF keeps fetching while ID is held, the instruction in IF/ID is overwritten and one instruction of the program is silently skipped - `retired` will come out too low and the register file will be wrong.

Checkpoint. `./rvsim tests/sample1_sum_loop.s --pipeline` must still report `cycles = 108, retired = 53, stalls = 40, flushes = 11`, and the register dump must still match the oracle. You have only moved code, so nothing may change.

6. Task 2 - The Forwarding Unit

File: `pipeline/hazard.c` (plus the EX stage of `pipeline.c`)

Stalling wastes cycles waiting for a value that **already exists**. The instruction in MEM has its ALU result sitting in `ex_mem.alu_result`; the instruction in WB has its final value sitting in `mem_wb.wb_val`. Neither has reached the register file yet, but both are readable at the start of the cycle. So route them straight into the ALU inputs:

```
              EX/MEM.alu_result ---+    (distance 1, highest priority)
              MEM/WB.wb_val -----+ |    (distance 2)
                                 v v
  ID/EX.rs1_val --------------> [mux] --> a --+--> ALU --> EX/MEM.alu_result
  ID/EX.rs2_val --------------> [mux] --> b --+
                                              +--> EX/MEM.store_val
```

Distance 3 needs no path at all: WB writes the register file in the same cycle ID reads it (Section 3).

The priority rule

If both latches match the same source register, **EX/MEM wins**. It holds the younger instruction, and the younger write is the one the consumer must see. Get this backwards and the following program puts 1 in `t1` instead of 2:

```
addi t0, zero, 1
addi t0, t0, 1      # t0 = 2 ; sits in EX/MEM when the next instruction is in EX
addi t1, t0, 0      # must see 2
```

Implementation

Add to `hazard.h`:

```
typedef struct {
    uint32_t a, b;        /* the operands EX should actually use */
    int fwd_ex, fwd_mem;  /* how many operands came from each path (stats) */
} Operands;

Operands forward_operands(const Pipeline *p, const IDEX *d);
```

In `hazard.c`, one block per operand - `a` from `rs1`, `b` from `rs2`:

```
Operands forward_operands(const Pipeline *p, const IDEX *d) {
    Operands o = { d->rs1_val, d->rs2_val, 0, 0 };
    if (!p->forwarding || d->fault != PF_NONE) return o;

    if (d->ctrl.uses_rs1 && d->instr.rs1 != 0) {
        if (produces(p->ex_mem.valid, &p->ex_mem.ctrl,
                     p->ex_mem.instr.rd, d->instr.rs1)) {
            o.a = p->ex_mem.alu_result;  o.fwd_ex++;      /* EX/MEM wins */
        } else if (produces(p->mem_wb.valid, &p->mem_wb.ctrl,
                            p->mem_wb.instr.rd, d->instr.rs1)) {
            o.a = p->mem_wb.wb_val;      o.fwd_mem++;
        }
    }
    /* ... the same block again for rs2 -> o.b ... */
    return o;
}
```

Make `produces()` non-static and declare it in `hazard.h` so both functions share it.

Four places must use the forwarded values

In the EX stage, replace every use of `d->rs1_val` / `d->rs2_val` with `o.a` / `o.b`. All four of these are easy to forget:

1. **ALU operands** - `out.alu_result = alu(&d->instr, d->pc, o.a, o.b);`
2. **Store data** - `out.store_val = o.b;` A store's value is captured in EX, so an unforwarded `store_val` writes a stale word to memory.
3. **Branch comparison** - `branch_taken(funct3, o.a, o.b)`. Otherwise a branch immediately after the instruction producing its operand is resolved on stale values and jumps the wrong way.
4. **`jalr` target** - `(o.a + imm) & ~1u`.

Wire up the mode

Add `int forwarding;` to `Pipeline`, set it in `pipeline_init(Pipeline *p, int forwarding)`, and add `--pipeline-forward` in `main.c` next to `--pipeline`.

Checkpoint. `./rvsim tests/sample1_sum_loop.s --pipeline-forward` must print 55, report `cycles = 68, retired = 53, stalls = 0, flushes = 11`, and produce a register dump identical to the oracle's.

7. Task 3 - Load-Use Hazard Detection

File: `pipeline/hazard.c`

Forwarding removes every stall except one, and it is removed by arithmetic, not by opinion. A load's value does not exist until the **end** of MEM, but the next instruction needs its operand at the **start** of EX - and those are the same cycle:

```
cycle ->          1     2     3     4     5
lw   t0, 0(gp)    IF    ID    EX    MEM   WB
                                    ^ t0 known at the END of cycle 4
```

```
addi t1, t0, 1          IF    ID    EX
                                    ^ t0 needed at the START of cycle 4
```

There is no path backwards in time, so the consumer must wait exactly one cycle. After that one stall it is in EX while the load is in WB, and the existing MEM/WB -> EX path delivers the value. One stall, and no second mechanism.

The detector

It runs in ID, in forwarding mode, and examines **only ID/EX**:

```
/* ... inside hazard_check(), in the p->forwarding branch ... */
if (p->id_ex.valid && p->id_ex.ctrl.mem_read && p->id_ex.instr.rd != 0 &&
    ((c->uses_rs1 && d->rs1 == p->id_ex.instr.rd) ||
     (c->uses_rs2 && d->rs2 == p->id_ex.instr.rd))) {
    v.stall = 1;
    v.load_use = 1;
}
return v;
```

Why only ID/EX? A load in EX/MEM finishes its memory access this cycle, so by the time the consumer is in EX the load is in WB and MEM/WB -> EX covers it. A load any further ahead has already written back. `mem_read` is the right signal to test, not the opcode - that is what you built the control unit for.

The stall actions are the same three as in Task 1, plus `nx.load_use_stalls++`.

Cases your detector must catch

All four of these are load-use hazards, because `uses_rs1` and `uses_rs2` already say which registers an instruction reads:

Program fragment	Consumer reads the loaded value as
`lw t0, 0(gp)` then `addi t1, t0, 1`	an ALU operand
`lw t0, 0(gp)` then `beq t0, zero, L`	a branch operand
`lw t0, 0(gp)` then `sw t0, 4(gp)`	store data (`rs2`)
`lw t0, 0(gp)` then `lw t1, 0(t0)`	an address base (`rs1`)

The third case is the one real processors optimise with a dedicated MEM -> MEM path. You are not building that: one stall plus MEM/WB -> EX gives the correct answer, and Section 11 asks you what the extra path would have saved.

Checkpoint. `./rvsim tests/sample4_data_hazards.s --pipeline-forward` must print 36 and report `cycles = 36, retired = 29, stalls = 2, flushes = 1`. Both remaining stalls are load-use stalls.

8. Task 4 - Instrumentation

File: `pipeline/pipeline_state.h`

A pipeline you cannot measure is a pipeline you cannot defend. Extend `Pipeline` with these counters and accumulate them inside `pipeline_cycle()`:

Counter	Incremented when	Counting rule
`stalls`	any cycle ID is held	once per stalled cycle, not once per hazard
`load_use_stalls`	a stall whose verdict had `load_use`	subset of `stalls`
`fwd_ex`	an operand is taken from EX/MEM	once per **operand**, so one instruction can add 2
`fwd_mem`	an operand is taken from MEM/WB	same rule
`retired`	a valid instruction leaves WB	never count bubbles
`flushes`	a taken branch or jump squashes ID	unchanged from Lab 2

Count a forward only when the control signal says the operand is used and the register number is non-zero - otherwise `lui` and `jal` inflate the totals. Print one summary line at the end of `pipeline_run_batch()`:

```
printf("pipeline: mode=%s cycles=%llu retired=%llu stalls=%llu flushes=%llu "
       "CPI=%.3f fwd_ex=%llu fwd_mem=%llu load_use=%llu\n",
       p->forwarding ? "forward" : "stall", ...,
       (double)p->cycles / (double)p->retired, ...);
```

The identity from Lab 2 must hold **in both modes, for every program**:

```
\text{cycles} = \text{retired} + \text{stalls} + \text{flushes} + 4
```

Every clock tick either retires an instruction, is lost to a stall, is lost to a flush, or is one of the four fill cycles before the first instruction reaches WB. If it does not hold, a counter is in the wrong place - and that is usually the same bug that is corrupting the register file.

9. Verification

Step 1 - the register files must match, in both modes

The single-cycle engine is the specification. Forwarding changes *when* a value is read, never *what* it is, so a correct forwarding pipeline is bit-identical to a correct stalling pipeline and to the oracle:

```
for f in tests/*.s; do
  ./rvsim $f --batch            > ref.txt
  ./rvsim $f --pipeline         > stall.txt
  ./rvsim $f --pipeline-forward > fwd.txt
  diff <(grep '^x' ref.txt) <(grep '^x' stall.txt) &&
  diff <(grep '^x' ref.txt) <(grep '^x' fwd.txt)   && echo "$f MATCH"
done
```

Diff only the `x...` register lines: the final `pc` legitimately differs because the pipeline's fetch PC runs ahead, and the cycle counts differ by design.

Step 2 - the numbers must be explainable

Copy `sample4_data_hazards.s` (Appendix A) and `micro.s` (Section 4) into `tests/`, then run all four programs in both modes. These are the reference figures for the policies specified in Tasks 1-3.

Stalling only (your Lab 2 baseline, unchanged by this lab):

Program	Instructions	Cycles	Stalls	Flushes	CPI
`micro.s`	5	15	6	0	3.000
`sample1_sum_loop.s`	53	108	40	11	2.038
`sample2_mem_and_branches.s`	13	26	8	1	2.000
`sample4_data_hazards.s`	29	69	35	1	2.379

With forwarding and load-use detection:

Program	Cycles	Stalls	Flushes	CPI	fwd EX/MEM	fwd MEM/WB	Cycles saved
`micro.s`	9	0	0	1.800	2	1	40%
`sample1_sum_loop.s`	68	0	11	1.283	13	11	37%
`sample2_mem_and_branches.s`	18	0	1	1.385	2	2	31%
`sample4_data_hazards.s`	36	2	1	1.241	9	7	48%

`retired` must equal the single-cycle cycle count exactly, in both modes - it is the number of instructions the program executes, and no hazard policy may change it. `stalls` and `flushes` may differ slightly from these tables if you made a defensible design choice; if they do, say which choice and why in your report. If `retired` is wrong, or the registers differ, you have a bug - not a design choice.

Step 3 - cycle-by-cycle debugging

When registers come out wrong, the final dump tells you nothing. Print one line per cycle at the bottom of `pipeline_cycle()` and compare it against the diagrams in Section 4:

```
fprintf(stderr, "c%-3llu IF=%04x ID=%04x EX=%04x MEM=%04x WB=%04x %s%s\n",
        (unsigned long long)nx.cycles, nx.if_id.pc, p->if_id.pc,
        p->id_ex.pc, p->ex_mem.pc, p->mem_wb.pc,
        stall ? "STALL " : "", flush ? "FLUSH" : "");
```

Run it on `micro.s` and check three things by hand: the diagonal appears; a stall repeats the same PC in ID while EX shows a bubble; and after a flush the next fetch is at the branch target.